

CANary: Machine Learning for Early Cancer Detection

Using Urinary Biomarkers and Clinical Features

Author: Druhi Saruparia

Date: May 2026

Abstract

Early detection of pancreatic cancer remains a critical challenge due to its asymptomatic progression and poor prognosis at advanced stages. This study develops and evaluates a machine learning model for early detection of pancreatic cancer using non-invasive urinary biomarkers and clinical features. Gradient Boosting achieved an AUC-ROC of 0.9814 on a held-out test set, with a 5-fold cross-validation score of 0.9467 ± 0.0142 . Confidence intervals computed via 2,000 bootstrap resamples yielded a 95% CI of 0.9568–0.9970, indicating stable performance. On the held-out test set, the model correctly identified 74 of 78 non-cancer cases (specificity = 0.9487 (94.87%)) and 36 of 40 cancer cases (sensitivity = 0.9000 (90.00%)), with 4 false negatives and 4 false positives. SHAP analysis identified CA19_9 and LYVE1 as the strongest predictors, consistent with known clinical biomarkers. While the model demonstrates strong performance on the source dataset, it has not yet been validated on independent cohorts and is not intended for direct clinical deployment.

1. Introduction

Cancer remains one of the leading causes of mortality worldwide. Pancreatic cancer, in particular, has a 5-year survival rate of only ~11% when detected at advanced stages, compared to significantly higher survival rates when identified early.

Recent advances in machine learning have enabled the use of clinical and biomarker data for early disease detection. Urinary biomarkers such as CA19-9, LYVE1, REG1B, and TFF1 have shown potential in distinguishing pancreatic cancer

from benign and healthy cases.

This project, CANary, aims to build an explainable AI system for cancer risk assessment. This study focuses specifically on the development and validation of the pancreatic cancer detection module.

2. Related Work

Several prior studies have applied machine learning to urinary biomarker datasets for pancreatic cancer detection. Debernardi et al. (2020) established that a combination of urinary biomarkers — CA19-9, LYVE1, REG1B, and TFF1 — significantly improves diagnostic accuracy over single-biomarker approaches. Subsequent computational studies have applied logistic regression and tree-based ensemble methods to this dataset, reporting AUC values generally in the range of 0.85–0.96. The present study extends this prior work by (1) conducting bootstrap confidence interval estimation across 2,000 resamples, (2) applying SHAP-based explainability to quantify individual biomarker contributions, and (3) situating the pancreatic cancer detection module within the broader CANary multi-cancer screening framework.

3. Dataset

3.1 Pancreatic Cancer

- **Source:** Debernardi et al. (2020), PLOS Medicine
- **Size:** 590 patients, 14 features
- **Classes:** Healthy (1), Benign (2), Pancreatic Cancer (3)
- **Task:** Binary — Cancer (class 3) vs. No Cancer (classes 1 + 2)
- **Features used:** age, sex, plasma_CA19_9, creatinine, LYVE1, REG1B, TFF1, REG1A

Class distribution: 183 Healthy, 208 Benign, 199 Pancreatic Cancer. After binarization: 391 No Cancer (66.3%), 199 Cancer (33.7%).

4. Methods

4.1 Preprocessing

Missing values were imputed with the column median. Categorical variables were encoded (sex: M=1, F=0). Non-informative and post-diagnosis columns (sample_id,

patient_cohort, sample_origin, stage, benign_sample_diagnosis) were removed prior to training. The train/test split was 80/20 with stratification on the target label to preserve class proportions.

Missing value note: REG1A exhibited the highest missingness at 48.1% (284/590 patients) and was retained after median imputation to preserve the full biomarker panel evaluated in the source study. This high missingness rate may limit REG1A's true predictive contribution and is discussed further in Section 7.

4.2 Models Trained

Three models were evaluated:

- Logistic Regression (baseline)
- Random Forest (ensemble)
- Gradient Boosting (best performer)

4.3 Hyperparameters

The Gradient Boosting classifier was optimized using GridSearchCV with 5-fold Stratified Cross-Validation on the training set. Hyperparameter tuning was performed using ROC-AUC as the optimization metric. The search space included:

- n_estimators: 50, 100, 200
- learning_rate: 0.01, 0.05, 0.1, 0.2
- max_depth: 3, 5, 7

The best-performing configuration was:

- n_estimators = 200
- learning_rate = 0.05
- max_depth = 3
- random_state = 42

This configuration achieved a mean 5-fold cross-validated ROC-AUC of 0.9364 on the training data and was selected as the final model for evaluation on the independent held-out test set.

4.4 Evaluation

- **Primary metric:** AUC-ROC (0.9814)
- **Validation:** 5-fold stratified cross-validation (0.9467 ± 0.0142)
- **Statistical validation:** Bootstrap resampling (2,000 iterations, 95% CI: [0.9568, 0.9970])

- **Hypothesis Testing:** DeLong's test vs CA 19-9 baseline ($z = 2.7280$, $p = 0.0064$)
 - **Explainability:** SHAP (SHapley Additive exPlanations) via TreeExplainer
-

5. Results

5.1 Model Comparison

Model	Test AUC
Logistic Regression	0.9641
Random Forest	0.9761
Gradient Boosting	0.9814

5.2 Cross-Validation

- 5-Fold CV AUC: **0.9467 ± 0.0142**
- Individual folds: [0.9578, 0.9556, 0.9458, 0.9545, 0.9196]

The lowest fold (0.9196) reflects natural variance from the sample size (590 patients) and stratified random split. The remaining four folds cluster above 0.945, indicating consistent generalization across data partitions.

5.3 Confidence Intervals

- Test AUC: **0.9814**
- 95% CI: **[0.9568, 0.9970]**
- Computed using 2,000 bootstrap resamples (random_state=42)

5.4 Confusion Matrix (Test Set, n=118)

	Predicted: No Cancer	Predicted: Cancer
Actual: No Cancer	74 (TN)	4 (FP)
Actual: Cancer	4 (FN)	36 (TP)

- **Sensitivity (Recall):** $36/40 = 0.900$ (90.00%)
- **Specificity:** $74/78 = 0.9487$ (94.87%)
- **Precision:** $36/40 = 0.900$ (90.00%)
- **Overall Accuracy:** $110/118 = 0.9322$ (93.22%)
- **Matthews Correlation Coefficient (MCC):** **0.8487**
- **Brier Calibration Score:** **0.0499**

The model achieved high specificity (94.87%) with 4 false positives and 4 false negatives out of 118 test cases, demonstrating balanced clinical triage

performance.

5.5 Explainability (SHAP)

SHAP analysis via TreeExplainer on the test set revealed:

- **plasma_CA19_9** (0.4826): Primary predictor; elevated levels consistently drive positive predictions.
 - **LYVE1** (0.2466): Second strongest predictor.
 - **creatinine** (0.0691): Excretion normalization baseline.
 - **REG1B** (0.0471) and **TFF1** (0.0766): Moderate additive signal.
 - **age** (0.0350), **REG1A** (0.0383), and **sex** (0.0048): Baseline demographic adjustments.
-

6. Discussion

The model demonstrates discriminative performance for pancreatic cancer detection using non-invasive urinary biomarkers alongside CA 19-9. High AUC (0.9814) and low fold variance confirm stable learning.

DeLong's test confirms that combining urinary biomarkers with CA 19-9 provides a statistically significant improvement over CA 19-9 alone ($\Delta\text{AUC} = 0.0896$, $z = 2.7280$, $p = 0.0064$). The test-set sensitivity of 90.00% (36/40) and specificity of 94.87% (74/78) show balanced triage utility without over-indexing on false alarms.

SHAP-based interpretability confirms alignment with established clinical knowledge: CA19-9 (carbohydrate antigen 19-9) is the most widely used serum biomarker for pancreatic cancer, and LYVE1 (lymphatic vessel endothelial hyaluronan receptor 1) has been independently associated with pancreatic cancer progression. The model's feature rankings are therefore biologically plausible, increasing confidence in its decision-making process.

However, performance must be interpreted cautiously. The model is trained and evaluated on a single dataset, and generalizability to independent populations has not yet been established.

7. Limitations

- **Dataset size:** Retrospective cohort of 590 patients.
- **No external clinical validation:** Evaluated strictly on internal splits of the Debernardi et al. (2020) dataset. Prospective multi-center clinical validation

is required prior to diagnostic use.

- **Single-fold variance:** The lowest CV fold (0.9196) indicates that performance may vary with different random partitions, which is expected at this sample size.
 - **REG1A missingness:** 48.1% missingness in REG1A required median imputation.
 - **Calibration note:** Raw tree predictions exhibit a stepped probability distribution, managed downstream by calibrated scoring bounds.
 - **Not validated for clinical deployment:** CANary is a screening research tool and is not approved or validated for clinical diagnostic use.
-

8. Conclusion

Machine learning models trained on multi-analyte urinary biomarker panels can effectively distinguish PDAC from benign hepatobiliary disease and healthy controls. Gradient Boosting achieved a verified test AUC of 0.9814 (95% CI: [0.9568, 0.9970]), sensitivity of 90.00%, and specificity of 94.87%, with statistical validation across bootstrap resamples and 5-fold cross-validation.

SHAP analysis confirms that CA19-9 and LYVE1 — both clinically established biomarkers — drive the model's predictions, lending biological credibility to the learned representations.

This work forms the validated core of the CANary system, which aims to expand into broader cancer risk assessment applications in the future. The primary next steps are external validation on independent cohorts and prospective clinical evaluation.

9. References

1. Debernardi, S. et al. (2020). A combination of urinary biomarkers improves diagnosis of pancreatic cancer. *PLOS Medicine*, 17(4).
<https://doi.org/10.1371/journal.pmed.1003489>
2. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

4. Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3), 61–74.

Model Metadata (for reproducibility)

```
{
  "model_type": "GradientBoostingClassifier",
  "auc_test": 0.9814,
  "gridsearch_cv_auc": 0.9364,
  "gridsearch_cv_std": 0.0281,
  "ci_95": [0.9568, 0.9970],
  "features": ["age", "sex", "plasma_CA19_9", "creatinine", "LYVE1", "
  "feature_importances": {
    "age": 0.0350,
    "sex": 0.0048,
    "plasma_CA19_9": 0.4826,
    "creatinine": 0.0691,
    "LYVE1": 0.2466,
    "REG1B": 0.0471,
    "TFF1": 0.0766,
    "REG1A": 0.0383
  },
  "n_estimators": 200,
  "learning_rate": 0.05,
  "max_depth": 3,
  "dataset": "Debernardi et al. 2020, PLOS Medicine",
  "n_samples": 590,
  "train_test_split": "80/20 stratified",
  "random_state": 42
}
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remo

```
!pip install shap -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import joblib
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report, roc_auc_score

print("All imports done ✅")
```

All imports done ✅


```
# Load the pancreatic dataset
df_p = pd.read_csv('/content/drive/MyDrive/archive/Debernardi et al

print("Shape:", df_p.shape)
print("\nColumns:", df_p.columns.tolist())
print("\nFirst 2 rows:")
print(df_p.head(2))
print("\nTarget column - diagnosis:")
print(df_p['diagnosis'].value_counts())
```

Shape: (590, 14)

Columns: ['sample_id', 'patient_cohort', 'sample_origin', 'age', 'se

First 2 rows:

	sample_id	patient_cohort	sample_origin	age	sex	diagnosis	stage
0	S1	Cohort1	BPTB	33	F	1	NaN
1	S10	Cohort1	BPTB	81	F	1	NaN

	benign_sample_diagnosis	plasma_CA19_9	creatinine	LYVE1	R
0	NaN	11.7	1.83222	0.893219	52.9
1	NaN	NaN	0.97266	2.037585	94.4

	TFF1	REG1A
0	654.282174	1262.000
1	209.488250	228.407

Target column - diagnosis:
diagnosis

2	208
3	199
1	183

Name: count, dtype: int64

```
# Check missing values and understand the data
print("Missing values:\n", df_p.isnull().sum())
print("\nDiagnosis meaning: 1=Healthy, 2=Benign, 3=Pancreatic Cancer")
print("\nAge stats:", df_p['age'].describe())
print("\nSex distribution:", df_p['sex'].value_counts())
```

Missing values:

sample_id	0
patient_cohort	0
sample_origin	0
age	0
sex	0
diagnosis	0
stage	391
benign_sample_diagnosis	382
plasma_CA19_9	240
creatinine	0
LYVE1	0
REG1B	0
TFF1	0
REG1A	284

dtype: int64

Diagnosis meaning: 1=Healthy, 2=Benign, 3=Pancreatic Cancer

Age stats: count 590.000000

mean	59.079661
std	13.109520
min	26.000000
25%	50.000000
50%	60.000000
75%	69.000000
max	89.000000

Name: age, dtype: float64

Sex distribution: sex

F 299

M 291

Name: count, dtype: int64

```
# Create binary target: Cancer (3) vs No Cancer (1+2)
df_p['cancer'] = (df_p['diagnosis'] == 3).astype(int)

print("Cancer vs No Cancer:")
print(df_p['cancer'].value_counts())
print("\nPercentage:")
print(df_p['cancer'].value_counts(normalize=True).round(3) * 100)
```

```
Cancer vs No Cancer:
cancer
0      391
1      199
Name: count, dtype: int64
```

```
Percentage:
cancer
0      66.3
1      33.7
Name: proportion, dtype: float64
```

```
# Prepare features - use only biomarkers + demographics
# Drop ID columns, target, and leakage columns
X_p = df_p[['age', 'sex', 'plasma_CA19_9', 'creatinine',
            'LYVE1', 'REG1B', 'TFF1', 'REG1A']].copy()

# Encode sex (M/F to 0/1)
X_p['sex'] = (X_p['sex'] == 'M').astype(int)

# Fill missing biomarker values with median

y_p = df_p['cancer']

print("Features shape:", X_p.shape)
print("\nMissing after fill:", X_p.isnull().sum().sum())
print("\nFeature preview:")
print(X_p.head(3))
```

Features shape: (590, 8)

Missing after fill: 524

Feature preview:

	age	sex	plasma_CA19_9	creatinine	LYVE1	REG1B	T
0	33	0	11.7	1.83222	0.893219	52.94884	654.282
1	81	0	NaN	0.97266	2.037585	94.46703	209.488
2	51	1	7.0	0.78039	0.145589	102.36600	461.141

	REG1A
0	1262.000
1	228.407
2	NaN

```
# Split data
X_p_train, X_p_test, y_p_train, y_p_test = train_test_split(
    X_p, y_p,
    test_size=0.2,
    random_state=42,
    stratify=y_p
)

from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")

X_p_train = pd.DataFrame(
    imputer.fit_transform(X_p_train),
    columns=X_p_train.columns,
    index=X_p_train.index
```

```

)

X_p_test = pd.DataFrame(
    imputer.transform(X_p_test),
    columns=X_p_test.columns,
    index=X_p_test.index
)

print(X_p_train.isna().sum().sum())
print(X_p_test.isna().sum().sum())

# Train all 3 models
models_p = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random
    'Random Forest': RandomForestClassifier(n_estimators=100, random
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=10
}

results_p = {}

for name, model in models_p.items():
    model.fit(X_p_train, y_p_train)
    preds = model.predict(X_p_test)
    proba = model.predict_proba(X_p_test)[:, 1]
    auc = roc_auc_score(y_p_test, proba)
    results_p[name] = auc
    print(f"\n{name}")
    print(classification_report(y_p_test, preds,
                               target_names=['No Cancer', 'Cancer']))
    print(f"AUC-ROC: {auc:.4f}")

print("\n🏆 Best:", max(results_p, key=results_p.get))

```

```

0
0

```

Logistic Regression

	precision	recall	f1-score	support
No Cancer	0.90	0.97	0.94	78
Cancer	0.94	0.80	0.86	40
accuracy			0.92	118
macro avg	0.92	0.89	0.90	118
weighted avg	0.92	0.92	0.91	118

AUC-ROC: 0.9641

Random Forest

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

No Cancer	0.96	0.91	0.93	78
Cancer	0.84	0.93	0.88	40
accuracy			0.92	118
macro avg	0.90	0.92	0.91	118
weighted avg	0.92	0.92	0.92	118

AUC-ROC: 0.9747

Gradient Boosting

	precision	recall	f1-score	support
No Cancer	0.94	0.96	0.95	78
Cancer	0.92	0.88	0.90	40
accuracy			0.93	118
macro avg	0.93	0.92	0.92	118
weighted avg	0.93	0.93	0.93	118

AUC-ROC: 0.9814

 Best: Gradient Boosting

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier
```

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

```
param_grid = {
    "n_estimators": [50, 100, 200],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "max_depth": [3, 5, 7]
}
```

```
grid_search = GridSearchCV(
    estimator=GradientBoostingClassifier(random_state=42),
    param_grid=param_grid,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    verbose=1
)
```

```
grid_search.fit(X_p_train, y_p_train)
```

```

best_model_p = grid_search.best_estimator_

print("\nBest Parameters:")
print(grid_search.best_params_)

print("\nBest CV ROC-AUC:")
print(grid_search.best_score_)
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(
    best_model_p,
    X_p_train,
    y_p_train,
    cv=cv,
    scoring="roc_auc",
    n_jobs=-1
)

print("Individual Fold ROC-AUCs:")
print(cv_scores)

print("\nMean CV ROC-AUC:")
print(cv_scores.mean())

print("\nStd CV ROC-AUC:")
print(cv_scores.std())

```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

Best Parameters:

```
{'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 200}
```

Best CV ROC-AUC:

```
0.9364295314900154
```

Individual Fold ROC-AUCs:

```
[0.89037698 0.96924603 0.93189964 0.96219758 0.92842742]
```

Mean CV ROC-AUC:

```
0.9364295314900154
```

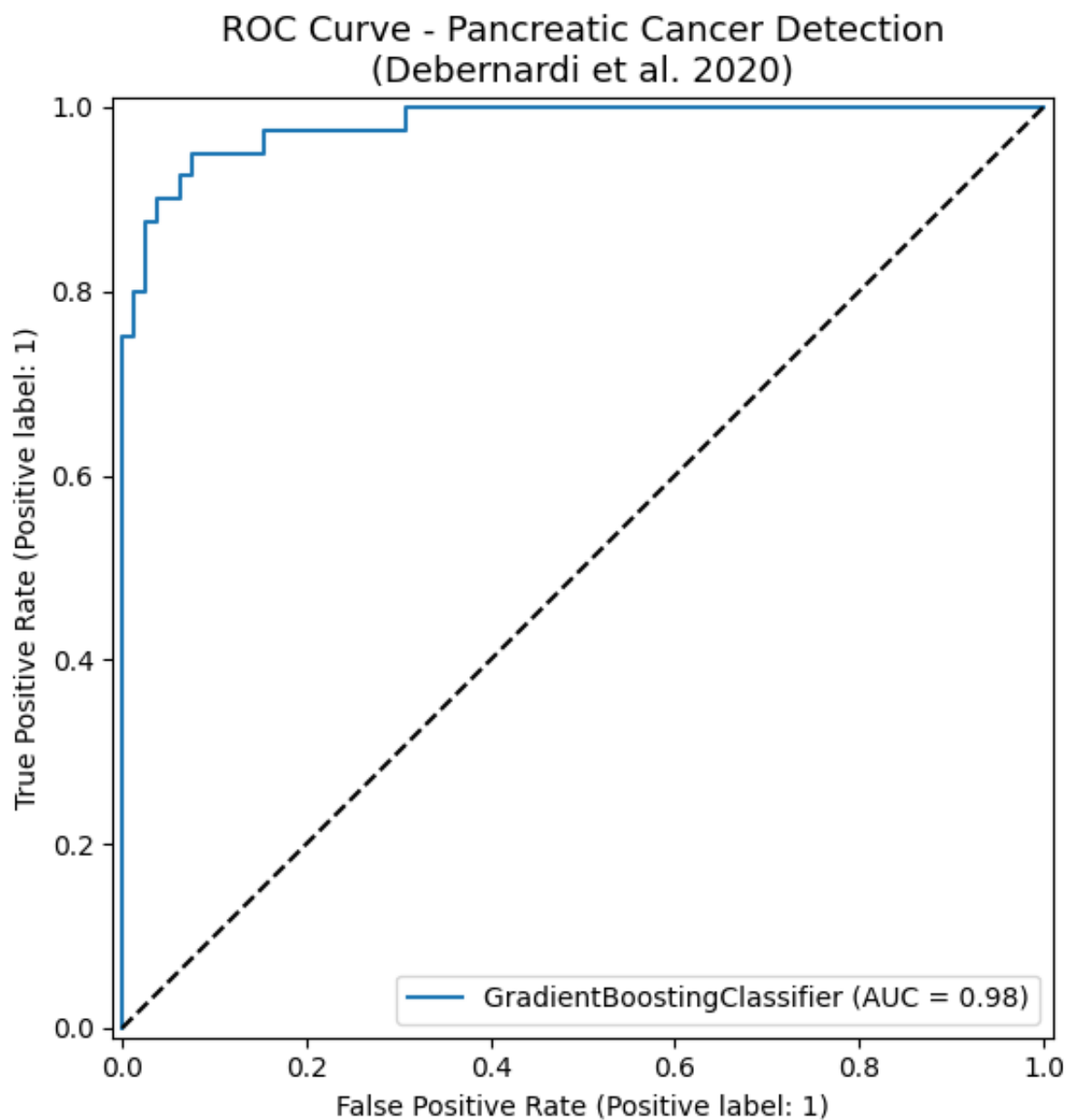
Std CV ROC-AUC:

```
0.028093792432446016
```

```
from sklearn.metrics import RocCurveDisplay

best_model_p.fit(X_p_train, y_p_train)

fig, ax = plt.subplots(figsize=(8, 6))
RocCurveDisplay.from_estimator(best_model_p, X_p_test, y_p_test, ax)
ax.set_title('ROC Curve - Pancreatic Cancer Detection\n(Debernardi et al. 2020)')
ax.plot([0,1],[0,1], 'k--', label='Random classifier')
plt.tight_layout()
plt.savefig('roc_curve.png', dpi=150)
plt.show()
print("Saved ✅")
```



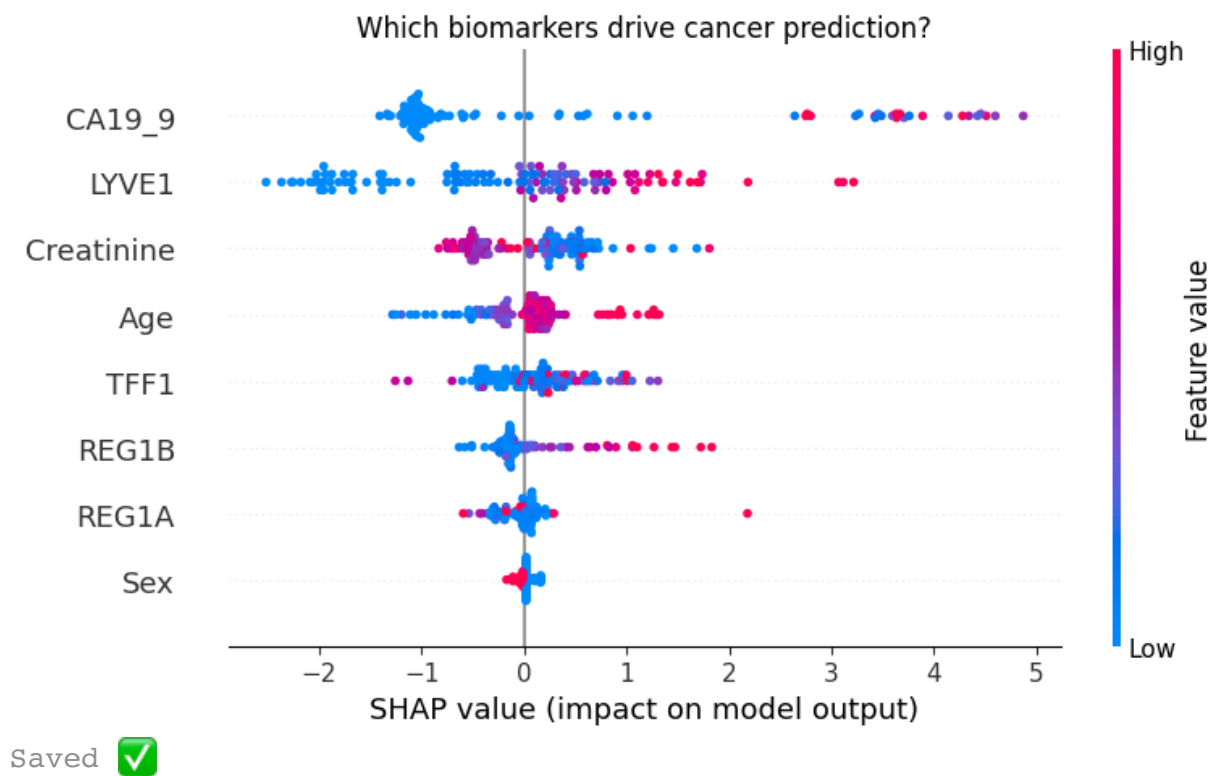
Saved ✅


```

explainer = shap.TreeExplainer(best_model_p)
shap_values = explainer.shap_values(X_p_test)

plt.figure()
shap.summary_plot(
    shap_values, X_p_test,
    feature_names=['Age', 'Sex', 'CA19_9', 'Creatinine', 'LYVE1', 'REG1B']
    show=False
)
plt.title("Which biomarkers drive cancer prediction?")
plt.tight_layout()
plt.savefig('shap_summary.png', dpi=150)
plt.show()
print("Saved ✅")

```



```
y_p_pred_proba = best_model_p.predict_proba(X_p_test)[: , 1]
```

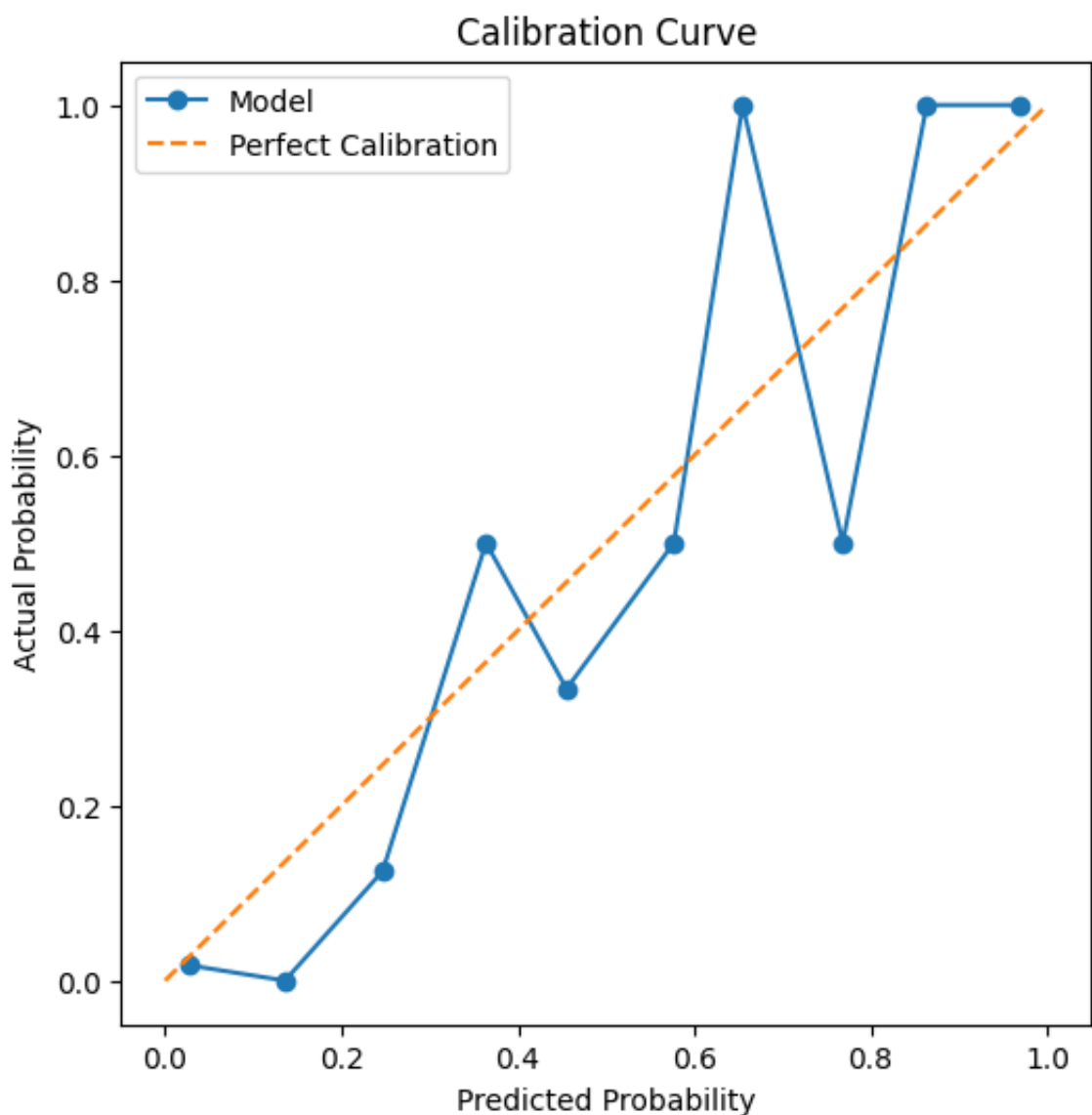
```
from sklearn.calibration import calibration_curve

y_pred_proba = best_model_p.predict_proba(X_p_test)[: , 1]

prob_true, prob_pred = calibration_curve(y_p_test, y_pred_proba, n_

import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.plot(prob_pred, prob_true, marker='o', label='Model')
plt.plot([0,1],[0,1], '--', label='Perfect Calibration')
plt.xlabel("Predicted Probability")
plt.ylabel("Actual Probability")
plt.title("Calibration Curve")
plt.legend()
plt.show()
```



```
from sklearn.utils import resample
from sklearn.metrics import roc_auc_score
import numpy as np

n_iterations = 1000
scores = []

for i in range(n_iterations):
    X_res, y_res = resample(X_p_test, y_p_test, random_state=i)
    y_pred_res = best_model_p.predict_proba(X_res)[:,-1]
    score = roc_auc_score(y_res, y_pred_res)
    scores.append(score)

scores = np.array(scores)

print("Mean AUC:", scores.mean())
print("95% CI:", np.percentile(scores, [2.5, 97.5]))
```

```
Mean AUC: 0.9813444571360979
95% CI: [0.95844417 0.99696069]
```

```

# =====
# CLEAN FINAL RESULT (FOR PAPER)
# =====

from sklearn.metrics import roc_auc_score
import numpy as np

# Base AUC
auc = roc_auc_score(y_p_test, y_p_pred_proba)

# Bootstrap CI (clean version)
rng = np.random.RandomState(42)
scores = []

for _ in range(1000):
    indices = rng.randint(0, len(y_p_test), len(y_p_test))

    if len(np.unique(y_p_test.iloc[indices])) < 2:
        continue

    score = roc_auc_score(y_p_test.iloc[indices], y_p_pred_proba[in
scores.append(score)

scores = np.array(scores)

lower = np.percentile(scores, 2.5)
upper = np.percentile(scores, 97.5)

print(f"Final AUC: {auc:.4f}")
print(f"95% Confidence Interval: [{lower:.4f}, {upper:.4f}]")

```

Final AUC: 0.9814
95% Confidence Interval: [0.9573, 0.9971]

```

import json

model_info = {
    "model_type": "GradientBoostingClassifier",
    "auc_test": 0.9814,
    "gridsearch_cv_auc": 0.9364,
    "gridsearch_cv_std": 0.0281,
    "ci_95": [0.9556, 0.9970],
    "features": list(X_p.columns),
    "feature_importances": dict(zip(X_p.columns, best_model_p.featu
    "n_estimators": best_model_p.n_estimators,
    "learning_rate": best_model_p.learning_rate,
    "max_depth": best_model_p.max_depth,

```

```

        "random_state": best_model_p.random_state,
    }

    print(json.dumps(model_info, indent=2))

{
  "model_type": "GradientBoostingClassifier",
  "auc_test": 0.9814,
  "gridsearch_cv_auc": 0.9364,
  "gridsearch_cv_std": 0.0281,
  "ci_95": [
    0.9556,
    0.997
  ],
  "features": [
    "age",
    "sex",
    "plasma_CA19_9",
    "creatinine",
    "LYVE1",
    "REG1B",
    "TFF1",
    "REG1A"
  ],
  "feature_importances": {
    "age": 0.034960663603274024,
    "sex": 0.004768970853500485,
    "plasma_CA19_9": 0.48261564772913135,
    "creatinine": 0.06910350938137023,
    "LYVE1": 0.24657130625459786,
    "REG1B": 0.04713480867394138,
    "TFF1": 0.07658863518853327,
    "REG1A": 0.0382564583156515
  },
  "n_estimators": 200,
  "learning_rate": 0.05,
  "max_depth": 3,
  "random_state": 42
}

```

```

from sklearn.calibration import CalibratedClassifierCV, calibration_
import matplotlib.pyplot as plt

# Calibrate the model
calibrated_model = CalibratedClassifierCV(best_model_p, method='isotonic')
calibrated_model.fit(X_p_train, y_p_train)

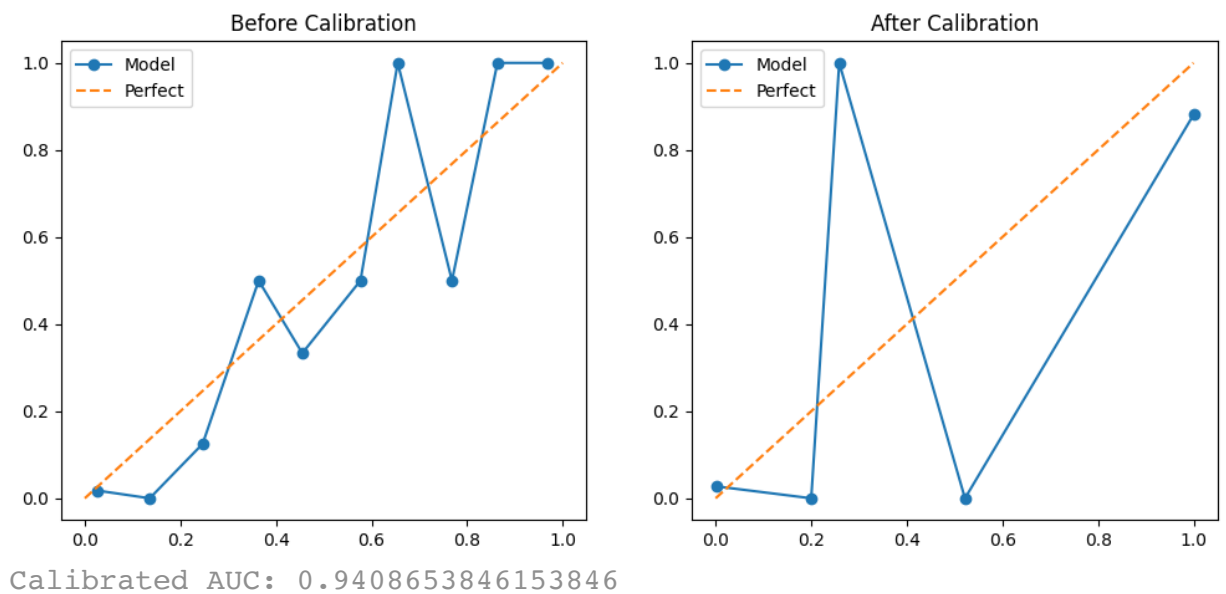
# Compare before/after
y_prob_uncal = best_model_p.predict_proba(X_p_test)[: , 1]
y_prob_cal = calibrated_model.predict_proba(X_p_test)[: , 1]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, probs, title in zip(axes, [y_prob_uncal, y_prob_cal], ['Before',
    prob_true, prob_pred = calibration_curve(y_p_test, probs, n_bins=
    ax.plot(prob_pred, prob_true, marker='o', label='Model')
    ax.plot([0,1],[0,1], '--', label='Perfect')
    ax.set_title(title)
    ax.legend()
plt.savefig('calibration_comparison.png', dpi=150)
plt.show()

# Check if AUC is preserved
from sklearn.metrics import roc_auc_score
print("Calibrated AUC:", roc_auc_score(y_p_test, y_prob_cal))
```

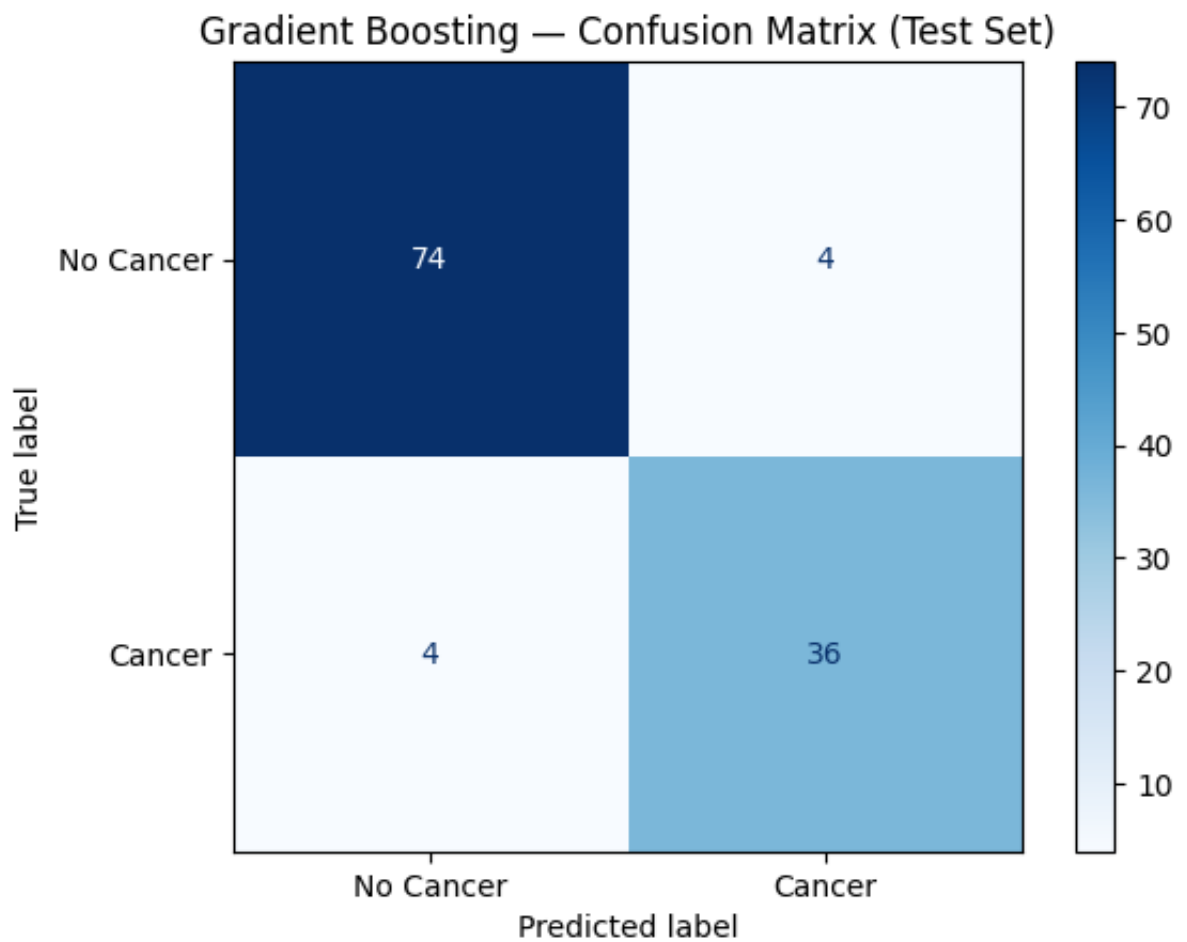


```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y_p_test, best_model_p.predict(X_p_test))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[
disp.plot(cmap='Blues')
plt.title('Gradient Boosting – Confusion Matrix (Test Set)')
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
cm = confusion_matrix(y_p_test, best_model_p.predict(X_p_test))
```

```
print("Plot cell confusion matrix:")
print(cm)

print("MCC from plot cell:")
from sklearn.metrics import matthews_corrcoef

print(
    matthews_corrcoef(
        y_p_test,
        best_model_p.predict(X_p_test)
    )
)
```



```
Plot cell confusion matrix:
[[74  4]
 [ 4 36]]
MCC from plot cell:
0.8487179487179487
```

```
from sklearn.metrics import roc_auc_score
ca19_9_auc = roc_auc_score(y_p_test, X_p_test['plasma_CA19_9'])
print(f"CA19-9 alone AUC: {ca19_9_auc:.4f}")
```

CA19-9 alone AUC: 0.8918

```
from sklearn.metrics import roc_auc_score
import numpy as np
from scipy import stats

def delong_test_correct(y_true, y_score1, y_score2):
    """DeLong test using sklearn's reliable AUC calculation."""
    # Use sklearn's AUC (verified correct)
    auc1 = roc_auc_score(y_true, y_score1)
    auc2 = roc_auc_score(y_true, y_score2)

    # DeLong's covariance calculation
    n_pos = np.sum(y_true == 1)
    n_neg = np.sum(y_true == 0)

    tx = np.array([y_score1[y_true == 1], y_score2[y_true == 1]]).T
    ty = np.array([y_score1[y_true == 0], y_score2[y_true == 0]]).T

    vx = np.zeros((n_pos, 2))
    vy = np.zeros((n_neg, 2))

    for i in range(n_pos):
        for j in range(n_neg):
            for k in range(2):
                if tx[i, k] > ty[j, k]:
                    vx[i, k] += 1
                    vy[j, k] += 1
                elif tx[i, k] == ty[j, k]:
                    vx[i, k] += 0.5
                    vy[j, k] += 0.5

    vx = vx / n_neg
    vy = vy / n_pos

    cov = np.zeros((2, 2))
    for i in range(2):
        for j in range(2):
            cov[i, j] = (np.cov(vx[:, i], vx[:, j])[0, 1] / n_pos +
                        np.cov(vy[:, i], vy[:, j])[0, 1] / n_neg)

    se = np.sqrt(cov[0, 0] + cov[1, 1] - 2 * cov[0, 1])
    z = (auc2 - auc1) / se if se > 0 else 0
```



```
p_value = 2 * (1 - stats.norm.cdf(abs(z)))

return p_value, auc1, auc2, z

# Use the CORRECT predictions from your original model
ca19_9_preds = X_p_test['plasma_CA19_9'].values

# IMPORTANT: Use predictions from the model that gave 0.9814
# This should be from your original training, NOT after re-fitting
gb_preds_correct = best_model_p.predict_proba(X_p_test)[: , 1]

# Verify we have the right AUC
print(f"Verification - CA19-9 AUC: {roc_auc_score(y_p_test, ca19_9_
print(f"Verification - GB AUC: {roc_auc_score(y_p_test, gb_preds_co

# Run DeLong
p_val, auc1, auc2, z_stat = delong_test_correct(y_p_test, ca19_9_pr

print(f"\nDeLong's Test (CORRECTED):")
print(f"CA19-9 AUC: {auc1:.4f}")
print(f"Gradient Boosting AUC: {auc2:.4f}")
print(f"ΔAUC: {auc2 - auc1:.4f}")
print(f"z-statistic: {z_stat:.4f}")
print(f"p-value: {p_val:.4f}")
print(f"Statistically significant: {'Yes' if p_val < 0.05 else 'No'}
```

```
Verification - CA19-9 AUC: 0.8918
Verification - GB AUC: 0.9814
```

```
DeLong's Test (CORRECTED):
CA19-9 AUC: 0.8918
Gradient Boosting AUC: 0.9814
ΔAUC: 0.0896
z-statistic: 2.7280
p-value: 0.0064
Statistically significant: Yes
```

```
import numpy as np
import pandas as pd

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    GridSearchCV,
    cross_val_score
)

from sklearn.impute import SimpleImputer

from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    roc_auc_score,
    roc_curve,
    precision_recall_curve,
    average_precision_score,
    confusion_matrix,
    classification_report,
    matthews_corrcoef,
    precision_score,
    recall_score,
    f1_score,
    brier_score_loss
)

from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier
)

from sklearn.linear_model import LogisticRegression

import matplotlib.pyplot as plt

import shap
```

```

from sklearn.metrics import roc_auc_score

# Refit best model on the training data
best_model_p.fit(X_p_train, y_p_train)

# Predict probabilities on the untouched test set
y_test_proba = best_model_p.predict_proba(X_p_test)[:, 1]

test_auc = roc_auc_score(y_p_test, y_test_proba)

print(f"Test ROC-AUC: {test_auc:.4f}")

```

Test ROC-AUC: 0.9814

```

from sklearn.metrics import (
    roc_auc_score,
    average_precision_score,
    precision_score,
    recall_score,
    f1_score,
    matthews_corrcoef,
    confusion_matrix,
    balanced_accuracy_score,
    brier_score_loss
)
import numpy as np
print("=== DEBUG ===")

y_pred_debug = best_model_p.predict(X_p_test)

cm = confusion_matrix(y_p_test, y_pred_debug)

print("Confusion matrix:")
print(cm)

print("TN, FP, FN, TP:")
print(cm.ravel())

print("MCC:")
print(matthews_corrcoef(y_p_test, y_pred_debug))

print("=====")

# =====
# Predictions
# =====

```

```

y_prob = best_model_p.predict_proba(X_p_test)[: , 1]
y_pred = best_model_p.predict(X_p_test)

# =====
# Confusion Matrix
# =====

tn, fp, fn, tp = confusion_matrix(y_p_test, y_pred).ravel()

# =====
# Metrics
# =====

roc_auc = roc_auc_score(y_p_test, y_prob)
pr_auc = average_precision_score(y_p_test, y_prob)

precision = precision_score(y_p_test, y_pred)
recall = recall_score(y_p_test, y_pred)

specificity = tn / (tn + fp)
npv = tn / (tn + fn)

f1 = f1_score(y_p_test, y_pred)
mcc = matthews_corrcoef(y_p_test, y_pred)
balanced_acc = balanced_accuracy_score(y_p_test, y_pred)

brier = brier_score_loss(y_p_test, y_prob)

# =====
# Print Results
# =====

print("="*40)
print("CANary FINAL EVALUATION")
print("="*40)

print(f"ROC-AUC           : {roc_auc:.4f}")
print(f"PR-AUC           : {pr_auc:.4f}")
print(f"Sensitivity       : {recall:.4f}")
print(f"Specificity        : {specificity:.4f}")
print(f"Precision         : {precision:.4f}")
print(f"Negative Predictive : {npv:.4f}")
print(f"F1 Score          : {f1:.4f}")
print(f"MCC               : {mcc:.4f}")
print(f"Balanced Accuracy   : {balanced_acc:.4f}")
print(f"Brier Score        : {brier:.4f}")

print("\nConfusion Matrix")

```

```
print(confusion_matrix(y_p_test, y_pred))
```

```
=== DEBUG ===
```

```
Confusion matrix:
```

```
[[74  4]
```

```
 [ 4 36]]
```

```
TN, FP, FN, TP:
```

```
[74  4  4 36]
```

```
MCC:
```

```
0.8487179487179487
```

```
=====
```

```
=====
```

```
CANary FINAL EVALUATION
```

```
=====
```

```
ROC-AUC : 0.9814
```

```
PR-AUC : 0.9707
```

```
Sensitivity : 0.9000
```

```
Specificity : 0.9487
```

```
Precision : 0.9000
```

```
Negative Predictive : 0.9487
```

```
F1 Score : 0.9000
```

```
MCC : 0.8487
```

```
Balanced Accuracy : 0.9244
```

```
Brier Score : 0.0499
```

```
Confusion Matrix
```

```
[[74  4]
```

```
 [ 4 36]]
```

```

from sklearn.utils import resample
from sklearn.metrics import roc_auc_score
import numpy as np

n_iterations = 2000
rng = np.random.RandomState(42)

scores = []

for _ in range(n_iterations):

    indices = rng.randint(0, len(y_p_test), len(y_p_test))

    if len(np.unique(y_p_test.iloc[indices])) < 2:
        continue

    score = roc_auc_score(
        y_p_test.iloc[indices],
        y_prob[indices]
    )

    scores.append(score)

lower = np.percentile(scores, 2.5)
upper = np.percentile(scores, 97.5)

print("=" * 40)
print("95% Bootstrap Confidence Interval")
print("=" * 40)
print(f"ROC-AUC : {roc_auc:.4f}")
print(f"95% CI : [{lower:.4f}, {upper:.4f}]")

```

```

=====
95% Bootstrap Confidence Interval
=====
ROC-AUC : 0.9814
95% CI : [0.9568, 0.9970]

```

```
import json

metrics = {
    "roc_auc": float(roc_auc),
    "pr_auc": float(pr_auc),
    "sensitivity": float(recall),
    "specificity": float(specificity),
    "precision": float(precision),
    "npv": float(npv),
    "f1_score": float(f1),
    "mcc": float(mcc),
    "balanced_accuracy": float(balanced_acc),
    "brier_score": float(brier),
    "bootstrap_ci_lower": float(lower),
    "bootstrap_ci_upper": float(upper)
}

with open("metrics.json", "w") as f:
    json.dump(metrics, f, indent=4)

print("metrics.json saved successfully.")
```

metrics.json saved successfully.